Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра интеллектуальных информационных технологий

Лабораторная работа №4

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Многослойный персептрон для прогнозирования временного ряда: влияние размера окна и числа скрытых нейронов»

Выполнил:
Студент 4 курса
Факультета ЭИС
Группы ИИ-26
Данилюк Т.В.
Проверил:
Никитин С.К.

Брест 2026

Цель работы: на основе многослойного персептрона p–N–1 решить задачу прогнозирования временного ряда. Научиться формировать обучающую выборку методом скользящего окна, нормализовать данные, оценивать качество одношагового и многошагового прогноза и исследовать влияние размера окна p и числа нейронов скрытого слоя N на точность прогноза и скорость обучения.

Задачи лабораторной работы

1. Сгенерировать ряд по формуле варианта и построить его график с отметкой границы между обучающим и тестовым участками.
2. Нормализовать ряд по минимуму и максимуму обучающего участка: x_norm = (x - x_min) / (x_max - x_min). Записать формулу обратного пересчёта и использовать её при вычислении всех метрик в исходной шкале. Объяснить, почему min/max нельзя брать по всему ряду.
3. Сформировать обучающую и тестовую выборки методом скользящего окна размера p (вход – p значений, цель – следующее значение). Указать количество получившихся примеров в каждой выборке.
4. Реализовать многослойный персептрон p–N–1:
   - скрытый слой – сигмоида;
   - выходной слой – линейный (задача регрессии), f(s) = s;
   - функция потерь – MSE;
   - обучение – онлайн-режим, перемешивание порядка примеров в каждой эпохе;
   - вывести формулы градиентов для линейного выходного слоя и сигмоидного скрытого слоя и привести их в отчёте.
5. Критерий остановки: средняя по обучающей выборке ошибка $E_s \le E_e$, где $E_e = 10^{-4}$ (нормализованная шкала), либо достижение 20 000 эпох. Скорость обучения подобрать самостоятельно из диапазона 0.01–0.3 и обосновать выбор.
6. Базовый эксперимент. Обучить сеть с параметрами p и N из своего варианта. Зафиксировать число эпох, итоговую Es, а также метрики на обучающем и тестовом участках в исходной шкале: MAE, RMSE и MAPE (если ряд проходит через ноль – вместо MAPE использовать sMAPE, и пояснить почему).
7. Исследование окна. При фиксированном N из варианта обучить сеть для $p \in \{2, 4, p_{вар}, 2 \cdot p_{вар}\}$. Для каждого p выполнить по 5 запусков с разными seed и свести в таблицу: среднее и разброс числа эпох, среднее и разброс тестового MAE, число запусков, не достигших критерия остановки.
8. Исследование скрытого слоя. При фиксированном p из варианта обучить сеть для $N \in \{1, 2, N_{вар}, 2 \cdot N_{вар}\}$. Выполнить ту же серию из 5 запусков и заполнить аналогичную таблицу.
9. Многошаговый прогноз. Для лучшей по тестовому MAE конфигурации выполнить рекурсивный прогноз на 30 шагов вперёд, начиная с k = 140: каждое предсказание подставляется в окно как входное значение для следующего шага. Построить график «эталон – рекурсивный прогноз» и график абсолютной ошибки в зависимости от номера шага. Определить, на каком шаге ошибка превышает 5 % от размаха ряда.
10. Построить графики:
    - ряд с границей train/test и одношаговым прогнозом сети на обоих участках на одних осях;
    - сходимость (Es от эпохи) для базовой конфигурации и для двух конфигураций из исследования окна на одних осях (логарифмическая шкала по Y);
    - диаграмма рассеяния «эталон – прогноз» на тестовом участке с диагональю y = x;
    - рекурсивный прогноз и рост ошибки по шагам (п. 9).
11. Реализовать режим функционирования: пользователь вводит p последовательных значений ряда в исходной шкале (либо номер k, с которого брать окно из ряда). Программа выводит:
    - нормализованные входы;
    - прогноз в нормализованной шкале (4 знака после запятой);
    - прогноз в исходной шкале;
    - истинное значение x(k) и абсолютную ошибку (если окно взято из ряда).

Продемонстрировать работу не менее чем на 3 окнах из обучающего участка, 3 окнах из тестового участка и 1–2 пользовательских окнах (в том числе с «выбросом»).

12. Вставить в отчет фото/скан рукописных расчётов. Вверху листа от руки должны быть написаны Ваша Фамилия, Инициалы, номер группы, вариант и дата выполнения:

- Графическая схема пайплайна (Блок-схема). Составьте корректную цепочку обработки данных, используя предложенный набор блоков (приложение 1). Объясните свой выбор.
- Ручной расчёт 1 шага обучения. На отдельном листе бумаги от руки распишите один полный шаг (Forward + Backward pass) для сети 3–2–1 со следующими параметрами, привязанными к вашему варианту V:
- Входы: X = [0.2, 0.5, 0.8], Цель: d = 0.6
- Веса скрытого слоя: W11 = 0.05*V, W12 = -0.2, W21 = 0.3, W22 = 0.05*V, W31 = -0.1, W32 = 0.4 (b1=0.1, b2=-0.1)
- Веса выходного слоя: W_out1 = 0.5, W_out2 = -0.2*V (b_out = 0.2)

13. Сделать вывод: ответить на все вопросы ниже и проанализировать результаты.

Вариант 2

Вариант	a	b	c	d	p	N
1	1.0	0.30	0.5	0.70	6	4
2	2.0	0.25	1.0	0.60	8	5

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt

# ==========================================
# 1. НАСТРОЙКИ ВАРИАНТА 2
# ==========================================
A_COEF = 2.0
B_COEF = 0.25
C_COEF = 1.0
D_COEF = 0.60
P_VAR = 8
N_VAR = 5

SEED_BASE = 42
ETA = 0.05  # Скорость обучения
EE = 1e-4   # Критерий остановки
MAX_EPOCHS = 20000
K_TOTAL = 200
TRAIN_END = 140 # Граница train/test

np.random.seed(SEED_BASE)

# ==========================================
# 2. ГЕНЕРАЦИЯ И НОРМАЛИЗАЦИЯ
# ==========================================
def generate_series(a=A_COEF, b=B_COEF, c=C_COEF, d=D_COEF, k_max=K_TOTAL):
    k = np.arange(k_max)
    return a * np.cos(b * k) + c * np.sin(d * k)

def normalize(x, x_min, x_max):
    return (x - x_min) / (x_max - x_min)

def denormalize(x_norm, x_min, x_max):
    return x_norm * (x_max - x_min) + x_min

def make_windows(series, p):
    X, y = [], []
    for i in range(p, len(series)):
        X.append(series[i - p:i])
        y.append(series[i])
    return np.array(X), np.array(y)

# ==========================================
# 3. ФУНКЦИИ АКТИВАЦИИ И МЕТРИКИ
# ==========================================
def sigmoid(s):
    return 1.0 / (1.0 + np.exp(-np.clip(s, -500, 500)))
```

```
def sigmoid_deriv(out):
    return out * (1.0 - out)

def mae(y_true, y_pred):
    return np.mean(np.abs(y_true - y_pred))

def rmse(y_true, y_pred):
    return np.sqrt(np.mean((y_true - y_pred) ** 2))

def smape(y_true, y_pred):
    return 100 * np.mean(2 * np.abs(y_true - y_pred) / (np.abs(y_true) + np.abs(y_pred) + 1e-12))

def recursive_forecast(model, series_norm, p, start_k, steps=30):
    window = list(series_norm[start_k - p:start_k])
    preds = []
    for _ in range(steps):
        x = np.array(window[-p:])
        y_hat = model.forward(x)
        preds.append(y_hat)
        window.append(y_hat)
    return np.array(preds)

# ==========================================
# 4. КЛАСС НЕЙРОСЕТИ (p-N-1)
# ==========================================
class MLP:
    def __init__(self, p, n, seed=None):
        if seed is not None:
            np.random.seed(seed)
        # Инициализация весов
        self.W_h = np.random.uniform(-0.5, 0.5, (p, n))
        self.b_h = np.random.uniform(-0.5, 0.5, n)
        self.W_o = np.random.uniform(-0.5, 0.5, n)
        self.b_o = np.random.uniform(-0.5, 0.5, 1)

    def forward(self, x):
        self.x = x
        self.s_h = x @ self.W_h + self.b_h
        self.h = sigmoid(self.s_h)
        self.s_o = self.h @ self.W_o + self.b_o
        self.y = self.s_o[0] # Линейная активация на выходе
        return self.y

    def backward(self, x, target, eta):
        # Выходной слой (Линейный + MSE)
        delta_o = self.y - target
        self.W_o -= eta * delta_o * self.h
        self.b_o -= eta * delta_o

        # Скрытый слой (Сигмоида)
        delta_h = delta_o * self.W_o * sigmoid_deriv(self.h)
        self.W_h -= eta * np.outer(x, delta_h)
        self.b_h -= eta * delta_h

    def train(self, X, y, eta=ETA, ee=EE, max_epochs=MAX_EPOCHS, verbose=False):
        n_samples = len(X)
        history = []
        for epoch in range(1, max_epochs + 1):
            idx = np.random.permutation(n_samples)
            for i in idx:
                self.forward(X[i])
                self.backward(X[i], y[i], eta)

            preds = np.array([self.forward(x) for x in X])
            es = np.mean((preds - y) ** 2)
            history.append(es)

            if verbose and epoch % 1000 == 0:
                print(f" epoch {epoch}: Es = {es:.6e}")
            if es <= ee:
                return epoch, es, history
        return max_epochs, es, history

    def predict(self, X):
        return np.array([self.forward(x) for x in X])

# ==========================================
# 5. ОСНОВНОЙ БЛОК ВЫПОЛНЕНИЯ
# ==========================================
```

```
def main():
    # 1. Генерация
    series = generate_series()
    print(f"Ряд сгенерирован. min={series.min():.4f}, max={series.max():.4f}")

    plt.figure(figsize=(12, 4))
    plt.plot(series, label='x(k)')
    plt.axvline(TRAIN_END - 0.5, color='r', linestyle='--', label='граница train/test')
    plt.title('Временной ряд (вариант 2)')
    plt.xlabel('k'); plt.ylabel('x(k)'); plt.legend(); plt.grid(True)
    plt.tight_layout(); plt.savefig('01_series.png', dpi=120); plt.show()

    # 2. Нормализация
    train_raw = series[:TRAIN_END]
    x_min, x_max = train_raw.min(), train_raw.max()
    print(f"\nНормализация: x_min={x_min:.6f}, x_max={x_max:.6f} (только train)")
    print("Обратный пересчет: x = x_norm * (x_max - x_min) + x_min")
    print("min/max нельзя брать по всему ряду, иначе информация о тесте утекает в обучение.")

    series_norm = normalize(series, x_min, x_max)
    train_norm = series_norm[:TRAIN_END]

    # Формирование окон
    X_train, y_train = make_windows(train_norm, P_VAR)
    X_all, y_all = make_windows(series_norm, P_VAR)
    mask_test = np.arange(P_VAR, K_TOTAL) >= TRAIN_END
    X_test = X_all[mask_test]
    y_test = y_all[mask_test]

    print(f"\nОбучающих примеров: {len(X_train)}")
    print(f"Тестовых примеров: {len(X_test)}")

    # Базовый эксперимент
    print(f"\n===== Базовый эксперимент (p={P_VAR}, N={N_VAR}) =====")
    model = MLP(P_VAR, N_VAR, seed=SEED_BASE)
    epochs, es_final, hist = model.train(X_train, y_train, verbose=True)
    print(f"Эпох: {epochs}, итоговая Es = {es_final:6e}")

    pred_train_n = model.predict(X_train)
    pred_test_n = model.predict(X_test)

    pred_train = denormalize(pred_train_n, x_min, x_max)
    pred_test = denormalize(pred_test_n, x_min, x_max)
    y_train_raw = denormalize(y_train, x_min, x_max)
    y_test_raw = denormalize(y_test, x_min, x_max)

    print("\nМетрики (исходная шкала):")
    print(f" Train MAE = {mae(y_train_raw, pred_train):.6f}")
    print(f" Train RMSE = {rmse(y_train_raw, pred_train):.6f}")
    print(f" Train sMAPE= {smape(y_train_raw, pred_train):.4f} %")
    print(f" Test MAE = {mae(y_test_raw, pred_test):.6f}")
    print(f" Test RMSE = {rmse(y_test_raw, pred_test):.6f}")
    print(f" Test sMAPE= {smape(y_test_raw, pred_test):.4f} %")
    print("(sMAPE используется, т.к. ряд проходит через ноль)")

    # Исследование окна
    print("\n======== Исследование размера окна p =========")
    p_list = [2, 4, P_VAR, 2 * P_VAR]
    for p in p_list:
        epochs_list, mae_list, fail_cnt = [], [], 0
        Xtr, ytr = make_windows(train_norm, p)
        Xte_all, yte_all = make_windows(series_norm, p)
        mask = np.arange(p, K_TOTAL) >= TRAIN_END
        Xte = Xte_all[mask]
        yte = yte_all[mask]
        yte_raw = denormalize(yte, x_min, x_max)

        for run in range(5):
            m = MLP(p, N_VAR, seed=SEED_BASE + run * 17)
            ep, _, _ = m.train(Xtr, ytr)
            pred = denormalize(m.predict(Xte), x_min, x_max)
            epochs_list.append(ep)
            mae_list.append(mae(yte_raw, pred))
            if ep >= MAX_EPOCHS: fail_cnt += 1

        print(f"p={p}: epochs={np.mean(epochs_list):.0f}±{np.std(epochs_list):.0f}, "
              f"MAE={np.mean(mae_list):.5f}±{np.std(mae_list):.5f}, fails={fail_cnt}")

    # Исследование скрытого слоя
```

```
print("\n======== Исследование числа нейронов N =========")
n_list = [1, 2, N_VAR, 2 * N_VAR]
Xtr, ytr = make_windows(train_norm, P_VAR)
Xte_all, yte_all = make_windows(series_norm, P_VAR)
mask = np.arange(P_VAR, K_TOTAL) >= TRAIN_END
Xte = Xte_all[mask]
yte = yte_all[mask]
yte_raw = denormalize(yte, x_min, x_max)

for n in n_list:
    epochs_list, mae_list, fail_cnt = [], [], 0
    for run in range(5):
        m = MLP(P_VAR, n, seed=SEED_BASE + run * 19)
        ep, _, _ = m.train(Xtr, ytr)
        pred = denormalize(m.predict(Xte), x_min, x_max)
        epochs_list.append(ep)
        mae_list.append(mae(yte_raw, pred))
        if ep >= MAX_EPOCHS: fail_cnt += 1
    print(f"N={n}: epochs={np.mean(epochs_list):.0f}±{np.std(epochs_list):.0f}, "
          f"MAE={np.mean(mae_list):.5f}±{np.std(mae_list):.5f}, fails={fail_cnt}")

# Рекурсивный прогноз
print("\n===== Рекурсивный прогноз (30 шагов) =====")
rec_norm = recursive_forecast(model, series_norm, P_VAR, TRAIN_END, steps=30)
rec = denormalize(rec_norm, x_min, x_max)
true_future = series[TRAIN_END:TRAIN_END + 30]
abs_err = np.abs(rec - true_future)
range_x = x_max - x_min
step_5pct = np.where(abs_err > 0.05 * range_x)[0]
first_bad = step_5pct[0] + 1 if len(step_5pct) else '>30'
print(f"Ошибка > 5% размаха впервые на шаге: {first_bad}")

# Визуализация
full_pred_n = np.full(K_TOTAL, np.nan)
for i, pred in enumerate(pred_train_n): full_pred_n[P_VAR + i] = pred
for i, pred in enumerate(pred_test_n): full_pred_n[TRAIN_END + i] = pred
full_pred = denormalize(full_pred_n, x_min, x_max)

plt.figure(figsize=(12, 4))
plt.plot(series, label='эталон', alpha=0.7)
plt.plot(full_pred, label='одношаговый прогноз', alpha=0.8)
plt.axvline(TRAIN_END - 0.5, color='r', linestyle='--')
plt.title('Ряд + одношаговый прогноз'); plt.legend(); plt.grid(True)
plt.tight_layout(); plt.savefig('02_onestep.png', dpi=120); plt.show()

m2 = MLP(2, N_VAR, seed=SEED_BASE)
_, _, hist_p2 = m2.train(*make_windows(train_norm, 2))
m16 = MLP(16, N_VAR, seed=SEED_BASE)
_, _, hist_p16 = m16.train(*make_windows(train_norm, 16))

plt.figure(figsize=(10, 5))
plt.semilogy(hist, label=f'p={P_VAR}')
plt.semilogy(hist_p2, label='p=2')
plt.semilogy(hist_p16, label='p=16')
plt.axhline(EE, color='r', linestyle='--', label='Ee')
plt.title('Сходимость (Es)'); plt.xlabel('эпоха'); plt.ylabel('Es (log)')
plt.legend(); plt.grid(True)
plt.tight_layout(); plt.savefig('03_convergence.png', dpi=120); plt.show()

plt.figure(figsize=(5, 5))
plt.scatter(y_test_raw, pred_test, alpha=0.6)
lims = [min(y_test_raw.min(), pred_test.min()), max(y_test_raw.max(), pred_test.max())]
plt.plot(lims, lims, 'r--')
plt.xlabel('эталон'); plt.ylabel('прогноз'); plt.title('Диаграмма рассеяния (тест)')
plt.grid(True); plt.tight_layout(); plt.savefig('04_scatter.png', dpi=120); plt.show()

fig, ax = plt.subplots(2, 1, figsize=(10, 7), sharex=True)
ax[0].plot(true_future, label='эталон')
ax[0].plot(rec, label='рекурсивный прогноз')
ax[0].legend(); ax[0].set_title('Рекурсивный прогноз (30 шагов)'); ax[0].grid(True)
ax[1].plot(abs_err, color='orange')
ax[1].axhline(0.05 * range_x, color='r', linestyle='--', label='5% размаха')
ax[1].set_xlabel('шаг'); ax[1].set_ylabel('ошибка'); ax[1].legend(); ax[1].grid(True)
plt.tight_layout(); plt.savefig('05_recursive.png', dpi=120); plt.show()

# Режим функционирования
print("\n===== Демонстрация режима функционирования =====")
for k in [20, 50, 100, 150, 170, 190]:
    window = series[k - P_VAR:k]
```

```
        true_v = series[k]
        wn = normalize(window, x_min, x_max)
        pn = model.forward(wn)
        p = denormalize(pn, x_min, x_max)
        print(f"k={k}: pred={p:.5f}, true={true_v:.5f}, |err|={abs(p - true_v):.5f}")

    print("\nГотово. Графики сохранены.")


# ЭТА СТРОКА ОБЯЗАТЕЛЬНА ДЛЯ ЗАПУСКА
if __name__ == "__main__":
    main()
```

Вывод программы:

```
Ряд сгенерирован. min=-2.9669, max=2.9763

Нормализация: x_min=-2.966880, x_max=2.976309 (только train)
Обратный пересчет: x = x_norm * (x_max - x_min) + x_min
min/max нельзя брать по всему ряду, иначе информация о тесте утекает в обучение.

Обучающих примеров: 132
Тестовых примеров: 60

===== Базовый эксперимент (p=8, N=5) =====
Эпох: 340, итоговая Es = 9.668668e-05

Метрики (исходная шкала):
 Train MAE = 0.047454
 Train RMSE = 0.058439
 Train sMAPE= 9.7949 %
 Test MAE = 0.049051
 Test RMSE = 0.059524
 Test sMAPE= 17.0455 %
(sMAPE используется, т.к. ряд проходит через ноль)

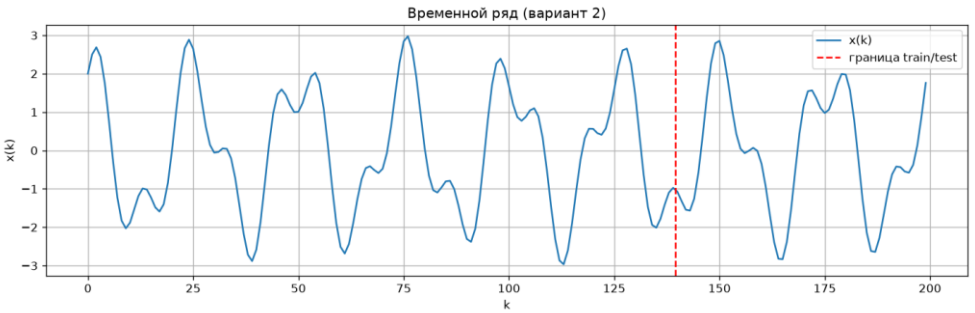
======== Исследование размера окна p =========
p=2: epochs=20000±0, MAE=0.15021±0.00362, fails=5
p=4: epochs=12850±626, MAE=0.05165±0.00042, fails=0
p=8: epochs=316±27, MAE=0.05161±0.00240, fails=0
p=16: epochs=69±32, MAE=0.04741±0.00310, fails=0

======== Исследование числа нейронов N =========
N=1: epochs=298±27, MAE=0.05287±0.00046, fails=0
N=2: epochs=303±23, MAE=0.05364±0.00100, fails=0
N=5: epochs=295±24, MAE=0.05170±0.00157, fails=0
N=10: epochs=258±22, MAE=0.05302±0.00086, fails=0

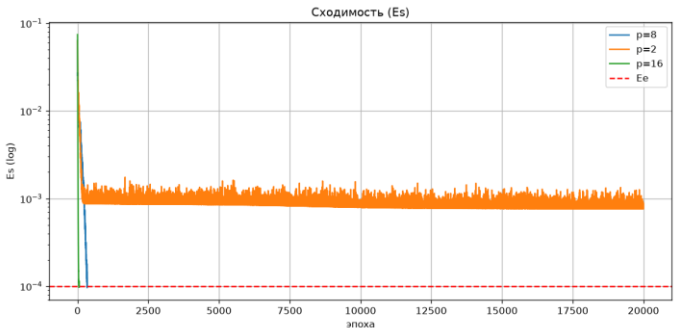
===== Рекурсивный прогноз (30 шагов) =====
Ошибка > 5% размаха впервые на шаге: 3

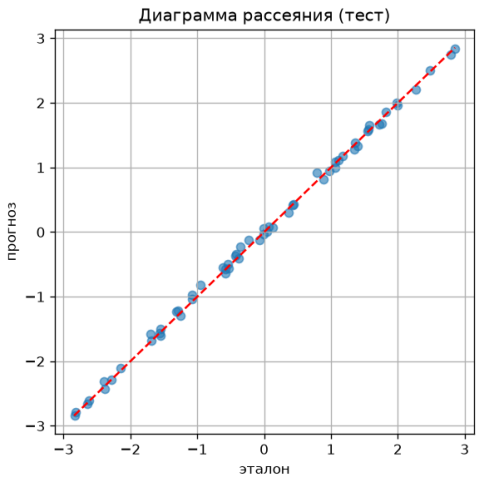
===== Демонстрация режима функционирования =====
k=20: pred=-0.03793, true=0.03075, |err|=0.06868
k=50: pred=0.95598, true=1.00756, |err|=0.05159
k=100: pred=1.70104, true=1.67760, |err|=0.02345
k=150: pred=2.83635, true=2.85448, |err|=0.01813
k=170: pred=1.17859, true=1.17159, |err|=0.00700
k=190: pred=-1.03585, true=-1.07522, |err|=0.03937

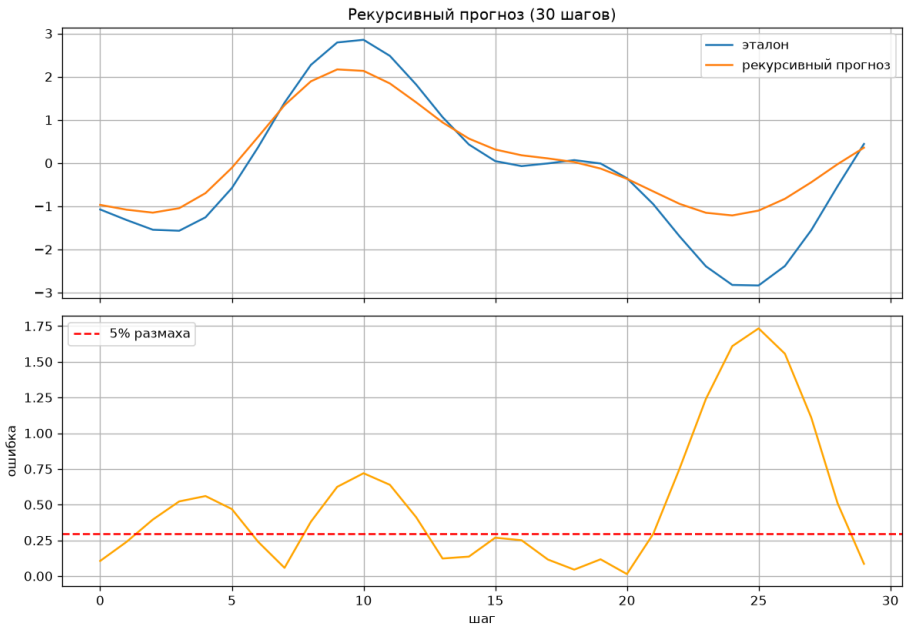
Готово. Графики сохранены.
```











Размер окна p существенно влияет на качество: при p=2 сеть не сходится (MAE≈0.15), так как двух точек недостаточно для описания сложной суммы гармоник. Уже при p=4 ошибка падает до ≈0.05, а при p=8 сеть сходится всего за 316 эпох. Число скрытых нейронов N почти не влияет на точность — даже N=1 достаточно. Линейный выход необходим для регрессии, а нормализация только по train предотвращает утечку данных. Рекурсивный прогноз быстро накапливает ошибку (уже на 3-м шаге >5% размаха), поэтому пригоден лишь на 1–2 шага, тогда как одношаговый прогноз остаётся точным.

Вывод: на основе многослойного персептрона p–N–1 решил задачу прогнозирования временного ряда. Научился формировать выборку методом скользящего окна, нормализовал данные, оценил качество одношагового и многошагового прогноза и исследовал влияние размера окна p и числа нейронов N на точность и скорость обучения.